Übungsblatt 6: Multicore-Echtzeit und Shared Memory

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Ziele und Ablauf

Sie begründen Kernrollen, Speicherlayout, Besitz und Sichtbarkeit. Grundlage sind Vorlesungen 1–6. AMP, Asymmetric Multiprocessing, bedeutet hier getrennte Firmwarekontexte; SMP, Symmetric Multiprocessing, eine gemeinsame Betriebssystemverwaltung mehrerer Kerne. MPU bedeutet Memory Protection Unit.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	20	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **85 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 75 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten. Im Papierkern sind 15 Minuten für Aufgabe 1, 15 Minuten für Aufgabe 2 und zusammen 20 Minuten für Aufgaben 3–4 vorgesehen.

Teil A: Papieraufgaben

Aufgabe 1: Softwaretrennung und Zeitbudget · 8 Punkte

1. Die M4-Firmware verwendet einen lokalen FreeRTOS-Kernel, die M7-Firmware eine eigene Hauptschleife. Ist dies das heutige AMP- oder ein SMP-Modell? Darf der M7 eine im gemeinsamen Speicher abgelegte M4-Queue-ID direkt verwenden? Schützt ein M4-Mutex automatisch vor M7-Zugriffen? Begründen Sie jede Aussage. (4 P)
2. Lehrmodell: Eine Reaktion benötigt 4 ms eigene M4-Arbeit, zusätzlich 3 ms Warten auf ein benötigtes M7-Ergebnis und 2 ms Businterferenz. Die Anteile sind ausdrücklich nicht überlappend. Relative Deadline: 8 ms. Berechnen Sie Antwortzeit und Fristabweichung. Warum reicht das Wissen um die eigene M4-Arbeit nicht aus? (2 P)
3. Beide Kerne konfigurieren verschiedene Pins desselben GPIO-Ports durch Lesen, Ändern und Zurückschreiben des gemeinsamen Konfigurationsregisters. Warum genügt die getrennte Pinzuordnung in CubeMX nicht? Nennen Sie eine geeignete Zuständigkeitsregel. (2 P)

Aufgabe 2: Layout, Start und Phasenbesitz · 8 Punkte

Gemeinsamer C-Typ aus der Vorlesung:

```
typedef struct {
    uint32_t magic;
    uint32_t state;
    uint32_t payload;
} ipc_block_t;
```

Jedes Wort hat vier Byte, natürliche Ausrichtung vier Byte. Das Objekt `ipc` wird auf 32 Byte ausgerichtet und seine Sektion auf 32 Byte gepolstert. SRAM4 beginnt bei `0x38000000` und besitzt 64 KiB. Die Kennung wird vor dem M4-Start gesetzt.

1. Geben Sie Feldoffsets, Typgröße, Sektionsreserve und MPU-Regionsgröße an. Warum sind die drei Größen verschieden? (2 P)
2. Die CM7-Map nennt `ipc` bei `0x38000000`, die CM4-Map bei `0x38000020`. Beide verwenden denselben Header. Ist das ein funktionsfähiger gemeinsamer Block? Was muss angepasst und geprüft werden? (2 P)
3. Beginnen Sie mit Payload null und `REQUEST_M4`. Bei jeder Antwort erhöht der M4 um eins. Vor der nächsten Anfrage erhöht der M7 ebenfalls um eins. Notieren Sie drei Antwortwerte, die beiden weiteren Anfragewerte und den endlichen Abschlusszustand. Nennen Sie das Schreibrecht je Phase und die Barrierepositionen. (2 P)
4. Eine Sektion ist `NOLOAD`, das M4-Startup nullt aber trotzdem einen überlappenden Bereich nach dem Aufwachen. Warum ist das falsch? Geben Sie die gültige Init-/Freigabereihenfolge und den Umgang mit einseitigem Core-Reset an. (2 P)

Aufgabe 3: Sichtbarkeit und ganze Cachezeilen · 8 Punkte

Die folgenden Werte sind synthetische Zustände, keine Boardmessungen. Der M7-Daten-Cache besitzt 32-Byte-Zeilen, der M4 kann ihn nicht direkt lesen.

1. M7 schreibt eins nur in eine Write-Back-Kopie, SRAM enthält weiterhin null. Welchen Wert kann M4 lesen? Umgekehrt schreibt M4 eins in SRAM, während M7 eine alte saubere gültige Kopie mit null hat. Was kann M7 lesen? Behebt `volatile` plus DMB beide Fälle? Begründen Sie. (4 P)
2. Ein Puffer beginnt bei `0x38000014` und hat 16 Byte Länge. Bestimmen Sie abgerundeten Wartungsbeginn, aufgerundetes exklusives Ende, Bytezahl und Zahl der berührten Zeilen. (2 P)
3. In einer Zeile liegen die Wörter A und B. Start beider null. M7 ändert in seiner Kopie A auf eins; die Zeile ist dirty, B dort noch null. M4 schreibt neun in B des SRAM. Was kann ein anschließendes Clean der alten M7-Zeile mit B tun? Welche Granularität muss die Besitzregel bei Cachewartung berücksichtigen? (2 P)

Aufgabe 4: MPU, Regionspriorität und Fehlerbudget · 8 Punkte

1. Region vier beschreibt SRAM4 ab `0x38000000` mit 64 KiB als Normal, TEX eins, C null, B null, S eins, Lesen/Schreiben und Execute Never. Region sechs überlappt die erste Hälfte als cachebares Normal-Write-Back mit S null. Beide Regionen und alle betreffenden Subregionen sind aktiviert. Welche Region gilt für `ipc` am Anfang? Erzeugt S eins einen M7/M4-Cacheabgleich? Schützt die M7-MPU damit automatisch vor M4- oder DMA-Zugriffen? (3 P)
2. Bestimmen Sie die letzte eingeschlossene Adresse der 64-KiB-Region und den Armv7-M-SIZE-Code mit Größe $= 2^{\mathrm{SIZE}+1}$ Byte. Welche Reihenfolge gilt zwischen MPU-Konfiguration, D-Cache-Aktivierung und Kommunikation? (2 P)
3. M4 wartet im Modul bis zu 2000 Kernel-Ticks, M7 prüft 2000 HAL-Millisekunden. Bei bestätigten 1000 Kernel-Ticks/s: Welche nominellen Budgets sind das? Warum beweisen sie keine zwei Sekunden vollständige IPC-Deadline? Darf der Fehlerpfad den Shared-Block einfach nullen? (3 P)

Transfer: Vier Aussagen im Entwurfsreview · 20 Minuten

Prüfen Sie zehn Minuten einzeln und besprechen Sie zehn Minuten. Keine zusätzlichen Punkte. Auf dem H755 werden ein nicht cachebarer Shared-Bereich und ein M4-Kernel verwendet. Es gibt keine automatische Schutzwirkung über beide Kerne.

1. „Nicht cachebar macht `counter++` zwischen M7 und M4 unteilbar.“ Widerlegen oder bestätigen Sie die Aussage mit einem Maschinenablauf ab Zählerstand 40.
2. „Ein lokaler M4-Mutex um den Zähler genügt auch für den Bare-Metal-M7.“ Welche Teilnahme fehlt?
3. „Nach einem Timeout darf M7 den ganzen Block neu initialisieren.“ Welche Besitz- und Wiederanlaufbedingung fehlt?
4. Eine obere Antwortgrenze von 7 ms enthält ausdrücklich schon 2 ms Busverzögerung. Ein Review addiert nochmals dieselben 2 ms. Ist 9 ms damit eine korrekt zerlegte Grenze? Was wäre zu tun, wenn die Buslastannahme geändert wird?

Teil B: Praxisfall · 15 Minuten

Praxisfall 6: Beide Sichten nebeneinander

Papieralternative, explizites Cachemodell: Der Phasenkanal hat genau einen Teilnehmer je Kern. `REQUEST_M4` übergibt an M4, `RESPONSE_M7` an M7. M7 besitzt eine gültige Write-Back-

Kopie; es findet keine Cachewartung statt. Nach der ersten Antwort und einer lokal geschriebenen Folgeanfrage liegt der folgende Zustand vor:

Sicht	state	payload
M7-Cache, Dirty-Kopie	REQUEST_M4	2
SRAM, daraus liest M4	RESPONSE_M7	1

1. Worauf warten beide Teilnehmer? Kann allein DMB die Differenz beheben? Belegt die SRAM-Debugansicht den Wert eines laufenden M7-Loads?
2. Vergleichen Sie mit einem korrekt nicht cachebaren, geordneten Kanal: Startpayload null, M4 erhöht pro Antwort um eins, M7 vor jeder Folgeanfrage ebenfalls. Welche drei Antworten und welcher finale Payload entstehen?
3. Was ist bei Timeout erlaubt? Weshalb wird die Integration eines neuen Linker-/MPU-Layouts nicht für diesen Erklärversuch verlangt?

Bei vorbereitetem Board: Die Lehrperson zeigt den nicht cachebaren Basislauf mit Antworten 1, 3, 5. Ein cachebarer Fehlerbuild ist nur mit eigener vorab geprüfter Projektvariante sinnvoll; der Fehler muss nicht in jedem Lauf sichtbar werden. Der Papierfall bleibt die verlässliche Erklärung.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 6; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Auflage; ST RM0399, UM2408, AN5617, AN4838 und AN5361; eingebundene STM32H7-HAL- und CMSIS-M7-Header sowie die Dokumentation der installierten STM32 VS Code Extension. Das öffentlich eingesehene H745-Boottemplate illustriert das Muster; die H755-Projektbasis und ihre Versionen bestimmen die tatsächliche Integration.